



write cursor rules to help me build this product: ReplyGuard AI — Build Plan

ROSTR pass on this doc: Receive (PRD.md) → Orchestrate (task order below) → Synthesize (agent harness executes) → Transform (Vercel deploy) → Return (live PWA URL)

Feed this file + PRD.md into your agent harness (Claude Code, Cursor, etc.) in order. Don't skip steps — each depends on the last existing.

Phase 3: Build (tonight, ~4-5 hrs with an agent harness)

Step 1 — Repo setup (15 min)

```
npx create-next-app@latest replyguard-ai --typescript --tailwind --app
```

```
cd replyguard-ai
```

```
npx shadcn@latest init
```

```
npx shadcn@latest add button input card avatar textarea
```

21st.dev chat UI (see PRD.md 5b)

```
npx shadcn@latest add https://21st.dev/r/tonyzebastian/chat-interface
```

```
npx shadcn@latest add https://21st.dev/r/ravikatiyar162/advanced-ai-chat-input
```

```
npx shadcn@latest add https://21st.dev/r/beratberkayg/ai-chat
```

```
npm install @supabase/supabase-js @supabase/ssr ai @openrouter/ai-sdk-provider @ai-sdk/amazon-bedrock zod
```

```
git init && git add . && git commit -m "init"
```

ai is the Vercel AI SDK core. @openrouter/ai-sdk-provider is your default gateway to DeepSeek/xAI/Gemini/Claude. @ai-sdk/amazon-bedrock is the separate enterprise/Hermes path — same interface, different provider.

Push to GitHub (gh repo create or manually via github.com) before continuing — Vercel deploys off the repo.

Step 2 — Supabase project (15 min)

Create project at supabase.com (free tier)

SQL editor → paste the schema from PRD.md section 6 → run

Enable Realtime on the messages table (Database → Replication)

Copy project URL + anon key

Step 3 — Auth (30 min)

Supabase Auth, email magic link only (no phone/Twilio needed)

File: app/(auth)/login/page.tsx — email input, "send magic link" button

File: lib/supabase/client.ts and lib/supabase/server.ts — standard SSR client setup

Test: sign up, click link in email, land on /chats

Step 4 — Conversation list (/chats) (30 min)

Protected route, redirect to /login if no session

List conversations for current user, most recent message preview

Empty state: "Start a conversation" if none exist

Step 5 — Thread view (/chats/[id]) (45 min)

iMessage-style bubbles (sender = right/blue, recipient = left/gray)

Subscribe to Supabase Realtime channel for this conversation — new rows in messages push instantly

Compose bar at bottom, plain textarea + send button (AI layer comes next)

Step 6 — AI compose layer (60-90 min, this is the core feature)

Vercel AI SDK's generateObject + Zod gives you structured output directly — no manual JSON parsing, and the model is swappable via one function.

lib/llm/models.ts:

```
import { createOpenRouter } from "@openrouter/ai-sdk-provider";
import { bedrock } from "@ai-sdk/amazon-bedrock";
```

```
const openrouter = createOpenRouter({ apiKey: process.env.OPENROUTER_API_KEY });
```

```
export function getModel(name = process.env.LLM_MODEL || "openrouter:deepseek/deepseek-chat") {
  const [provider, ...rest] = name.split(":");
  const modelId = rest.join(":");
  if (provider === "bedrock") return bedrock(modelId);
  return openrouter(modelId);
}
```

lib/llm/rewrite.ts:

```
import { generateObject } from "ai";
import { z } from "zod";
import { getModel } from "../models";
```

```
const RewriteSchema = z.object({ options: z.array(z.string()).length(3) });
```

```
export async function rewriteMessage(draft: string, contactRules: string, history: string) {
  const { object } = await generateObject({
    model: getModel(),
    schema: RewriteSchema,
    system: You rewrite text messages to fit the sender's rules for this contact. Contact
    rules: ${contactRules} Recent conversation context: ${history},
    prompt: draft,
  });
  return object.options;
}
```

app/api/ai/rewrite/route.ts:

```
import { rewriteMessage } from "@lib/llm/rewrite";
```

```
export async function POST(req: Request) {
  const { draft, contactRules, recentHistory } = await req.json();
```

```
const options = await rewriteMessage(draft, contactRules, recentHistory);
return Response.json({ options });
}
```

Swap models any time via the LLM_MODEL env var: openrouter:deepseek/deepseek-chat, openrouter:x-ai/grok-2, openrouter:google/gemini-2.0-flash, openrouter:anthropic/claude-sonnet-4-6, or bedrock:... — zero code changes.

UI: use the AI Chat option-picker pattern from @beratberkayg's component (installed in Step 1) — user types draft → taps "AI check" → shows 3 option cards + "send as-is" + edit-inline option → tap one → sends

Store both content (final sent) and ai_original_draft on the message row for later analytics (v1.1)

Step 7 — Contact profile (/chats/[id]/profile) (30 min)

Form: tone notes, no-send rules (free text, e.g. "no messages after 9pm"), relationship notes

Saves to contacts table, read by the rewrite API on every send

Step 8 — Local test (15 min)

Two browser windows, two test accounts, send messages back and forth

Confirm: realtime delivery works, AI rewrite returns 3 options, contact rules actually change tone

Phase 4: Deploy (30 min)

git push origin main

vercel.com → New Project → import repo

Env

vars: NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY, OPENROUTER_API_KEY, LLM_MODEL(default openrouter:deepseek/deepseek-chat)

Deploy

Add PWA manifest (app/manifest.ts in Next.js 15) so it's installable on iPhone home screen — this is your "TestFlight substitute" for tonight

Test the live URL end to end on your actual phone: sign up, add a contact, send a rewritten message, confirm realtime delivery

Done tonight = live URL + installable on your phone + one real message sent through the full AI flow.

Phase 2 (later, needs a little money): real SMS

Twilio account (\$15 free trial credit), buy a number

New API route sends via Twilio's messages.create after the AI rewrite step

Recipient without the app receives a real text from your Twilio number — not blue-bubble iMessage, and not "from" the sender's own number, but real-world delivery

This is the point where a \$99/yr Apple Developer account becomes worth it too, if you want push notifications + App Store distribution instead of PWA

Phase 3 (later): native iOS + TestFlight

Wrap the PWA in Capacitor or rebuild critical screens in SwiftUI

Apple Developer Program (\$99/yr) required for TestFlight

Add push notifications (APNs) so it feels like real texting, not "open the app to see if you got a reply"

Scope discipline

If you catch yourself adding anything not in PRD.md section 4 before tonight's deploy — stop, it's v1.1. FPE: finish this version first.

Here are your Cursor Rules — drop this as `.cursor/rules/replyguard.mdc` in your project root before you paste Prompt 0:[¹⁻¹1_2][[^]1_3]

```
---
title: ReplyGuard AI — Cursor Rules
description: Governing rules for the Cursor agent building ReplyGuard AI
alwaysApply: true
---

## Identity & Mission

You are building ReplyGuard AI — a real-time 1:1 messaging PWA with an AI
compose layer that rewrites drafts before sending, based on per-contact rules.
The full spec lives in `PRD.md`. The build order lives in `BUILD_PLAN.md`.
Read both files before writing a single line of code.

---

## Stack — Locked. Do Not Deviate.

- Framework: Next.js 15, App Router, TypeScript strict mode
- Styling: Tailwind CSS only — no inline styles, no CSS modules, no styled-components
- Components: shadcn/ui base + three 21st.dev components (chat-interface, advanced-search, profile)
- Database / Auth / Realtime: Supabase only — no Firebase, no PlanetScale, no Prisma
- AI Layer: Vercel AI SDK (`ai` package) with `generateObject` + Zod. Provider switcher
- Deployment: Vercel (Next.js) — no Docker, no custom server

If you think a different tool is better, write it to `NOTES.md` under `## Agent suggestions`

---

## File & Folder Conventions
```

```
app/
(auth)/login/page.tsx ← magic link auth page
chats/page.tsx ← conversation list
chats/[id]/page.tsx ← thread view
chats/[id]/profile/page.tsx ← contact rules form
api/
ai/rewrite/route.ts ← POST /api/ai/rewrite
messages/send/route.ts ← POST /api/messages/send
conversations/route.ts ← GET /api/conversations
contacts/[id]/route.ts ← GET + PUT /api/contacts/:id
lib/
llm/
models.ts ← getModel() provider switcher
rewrite.ts ← rewriteMessage() with generateObject
```

supabase/
client.ts ← browser Supabase client (@supabase/ssr)
server.ts ← server Supabase client (@supabase/ssr)
components/
ui/ ← shadcn + 21st.dev components only
NOTES.md ← v1.1 backlog + agent suggestions

```
Never create files outside this structure without asking first.
```

```
---
```

```
## API Routes – Thin Handlers Only
```

- API routes (`app/api/\*\*/route.ts`) must contain **no business logic**
- Parse the request, call one lib function, return the response – that's it
- All LLM logic lives in `lib/llm/`, all DB logic lives in `lib/supabase/`
- Example correct pattern:

```
```typescript
// app/api/ai/rewrite/route.ts
import { rewriteMessage } from "@lib/llm/rewrite";

export async function POST(req: Request) {
 const { draft, contactRules, recentHistory } = await req.json();
 const options = await rewriteMessage(draft, contactRules, recentHistory);
 return Response.json({ options });
}
```

## AI Layer Rules

- Always use `generateObject` from the `ai` package — never `generateText` for the `rewrite` endpoint
- The Zod schema must produce exactly 3 options: `z.object({ options: z.array(z.string()).length(3) })`
- Never hardcode a model name — always call `getModel()` from `lib/llm/models.ts`
- Provider is selected by the `LLM_MODEL` env var: `openrouter:<model-id>` or `bedrock:<model-id>`
- Default model when `LLM_MODEL` is unset: `openrouter:deepseek/deepseek-chat`
- Do not add retry logic, streaming, or caching to the `rewrite` call in v1 — keep it a single `generateObject` call

```
// lib/llm/models.ts – exact implementation, do not refactor
import { createOpenRouter } from "@openrouter/ai-sdk-provider";
import { bedrock } from "@ai-sdk/amazon-bedrock";

const openrouter = createOpenRouter({ apiKey: process.env.OPENROUTER_API_KEY });
```

```
export function getModel(name = process.env.LLM_MODEL || "openrouter:deepseek/deepseek") {
 const [provider, ...rest] = name.split(":");
 const modelId = rest.join(":");
 if (provider === "bedrock") return bedrock(modelId);
 return openrouter(modelId);
}
```

## Supabase Rules

- Use `@supabase/ssr` for both client and server — never `@supabase/supabase-js` directly in components
- RLS is enabled on all four tables (users, contacts, conversations, messages) — never disable it
- Realtime subscription must use a named channel per conversation:  
`supabase.channel('conversation-<id>')`
- Subscribe in a `useEffect`, always return the unsubscribe cleanup function
- Never query the database from a Client Component — use Server Components or API routes
- Do not ask for or invent Supabase credentials — ask the user if `.env.local` is missing them

## Auth Rules

- Email magic link only — no password flow, no OAuth, no phone auth
- Protect `/chats` and all sub-routes with a middleware or layout check that redirects to `/login` if no session
- Use `supabase.auth.getSession()` in Server Components,  
`supabase.auth.onAuthStateChange()` in Client Components

## Database Schema — Exact, Do Not Alter

Use the exact schema from `PRD.md` section 6. Do not rename columns, add columns, change types, or remove tables. If you believe a schema change is needed, write it to `NOTES.md` under `## Schema change requests` and use the existing schema for now.

## UI Rules

- iMessage-style bubbles: sender messages right-aligned with blue background, recipient messages left-aligned with gray background
- Use the `chat-interface` component from `@tonyzebastian` for the thread shell
- Use `advanced-ai-chat-input` from `@ravikatiyar162` for the compose bar — the "AI check" button lives here

- Use the option-picker pattern from ai-chat by @beratberkayg for displaying the 3 rewrite options
- The 3 rewrite option cards must also include a "send as-is" button and an inline edit option
- Timestamps on every message bubble
- Empty state on /chats: centered text "Start a conversation" — no illustration needed for v1

## Scope Enforcement (FPE — Finish Previous version Early)

MVP scope = PRD.md section 4 only. The 5 locked features are:

1. Email magic link auth
2. Real-time 1:1 chat
3. AI compose layer (draft → 3 rewrite options → send)
4. Per-contact rules (tone notes, no-send rules, relationship notes)
5. iMessage-style UI

**Do not build, scaffold, or stub any of the following in v1:**

- Analytics dashboard
- Conversation summaries
- Group chats
- Native iOS / Capacitor wrapper
- Real SMS via Twilio
- Push notifications (APNs)
- Apple Sign In
- Typing indicators beyond basic UI placeholder
- Python/FastAPI rules microservice

If you catch yourself building any of the above, **stop immediately**, add it to NOTES.md under ## v1.1 backlog, and return to the current BUILD\_PLAN.md step.

## Build Order — Sequential, No Skipping

Execute BUILD\_PLAN.md steps in this exact order. Do not start a step until the previous step's acceptance check passes:

1. Repo scaffold + 21st.dev components
2. Supabase schema + realtime
3. Auth (magic link)
4. Conversation list /chats
5. Thread view /chats/[id] + realtime delivery

6. AI compose layer (the core feature)
7. Contact profile /chats/[id]/profile + rules
8. Local end-to-end test
9. PWA manifest + Vercel deploy

**After each step:** run the app, confirm no errors, tell the user what to manually test, then **stop** and wait for explicit go-ahead before continuing.

## Environment Variables

Required in `.env.local` (never invent placeholder values — ask the user):

```
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
OPENROUTER_API_KEY=
LLM_MODEL=openrouter:deepseek/deepseek-chat
```

Never commit `.env.local` to git. Confirm `.gitignore` includes it before the first commit.

## Commit Discipline

Commit after each `BUILD_PLAN.md` step with a semantic message:

- `init: scaffold + ui components`
- `feat: supabase schema + realtime`
- `feat: auth magic link`
- `feat: conversation list`
- `feat: thread view + realtime`
- `feat: ai compose layer`
- `feat: contact profile + rules`
- `fix: e2e test fixes`
- `feat: pwa manifest + deploy`

No squash commits. One commit per step so rollbacks are clean.

## Agent Behavior Contract

- If the user doesn't respond within a step, **do not proceed** to the next step
- If you encounter an ambiguity in the PRD or `BUILD_PLAN`, **ask one specific question** — don't guess
- If a 21st.dev component doesn't install or renders broken, report it immediately and ask before substituting

- Never rename, reorganize, or "clean up" existing working code unless the user explicitly requests it
- NOTES.md is your escape valve — anything you can't do within these rules goes there

\*\*\*

## ## How to Use This

**\*\*Placement:\*\*** Save as ``.cursor/rules/replyguard.mdc`` in the project root. Cursor will

**\*\*Activation order:\*\***

1. Drop ``PRD.md``, ``BUILD_PLAN.md``, and ``BUILD_PROMPTS.md`` in the repo root<sup>[^1\_2][^1\_3]</sup>
2. Add the ``.cursor/rules/replyguard.mdc`` file above
3. Open Cursor, go to composer/agent mode
4. Paste **\*\*Prompt 0\*\*** from ``BUILD_PROMPTS.md`` to kick off — the rules file will already

The rules file does three things your prompts alone can't: they **\*\*persist across turns**

`<div align="center">✱</div>`

[^1\_1]: BUILD\_PLAN.md

[^1\_2]: PRD.md

[^1\_3]: BUILD\_PROMPTS.md

[^1\_4]: ROSTR-A-Unified-Architecture-for-Production-Grade-Multi-Agent-Systems-with-P

[^1\_5]: 02\_RAGDAL\_Whitepaper.md

[^1\_6]: 02\_RAGDAL\_Whitepaper.pdf

[^1\_7]: Soul.md template to include with PAL Rostr etc. cr.pdf

[^1\_8]: Master SOUL.md Template (for building other agents).docx

[^1\_9]: JTBD (Jobs-to-be-Done) Agent.pdf

[^1\_10]: Today!!!\_Build so newsletter \_Build music industry.pdf

[^1\_11]: Civic pie website map\_-- MyPie (newsfeed - data,.pdf

[^1\_12]: I'm building a master spreadsheet with each precin.pdf

[^1\_13]: Let's build an initiative like 48th ward dems, but.pdf

[^1\_14]: Let's build an initiative like 48th ward dems, but (1).pdf

[^1\_15]: RAG DAL (Dynamic Acquisition Layer) Agent\_\_The RAG.pdf

[^1\_16]: rewrite this NPAO\_ Necessity, Priority, Anxiety, O.pdf

[^1\_17]: cleanthis up # RAG DAL â€” Dynamic Acquisition Lay.pdf

[^1\_18]: Ideas from video\_Artist FACTS \_Instructions stop.pdf

[^1\_19]: what is an opensource aws.pdf

[^1\_20]: cleanthis up # RAG DAL â€” Dynamic Acquisition Lay (1).pdf

---

# great. now how do I add this to my cursor chat

There are two ways to add the rules file — pick whichever matches how you prefer to v

## ## Option 1 — File Drop (Recommended, Persists Across Sessions)

This is the proper method. The rules auto-attach to every Cursor agent/composer sess:

**\*\*Steps:\*\***

1. In your project root (``replyguard-ai/``), create the folder:

```
```bash
mkdir -p .cursor/rules
```

2. Create the file:

```
touch .cursor/rules/replyguard.mdc
```

3. Paste the full rules content I gave you into that file and save it

4. Open Cursor — the rules are now active automatically for every agent session in this project^[1]

The `.mdc` extension with the `alwaysApply: true` frontmatter (already in the rules I wrote) means Cursor loads it without you having to reference it manually.^[2]

Option 2 — Cursor Settings UI (Global, No File Needed)

If you want to set it once for all projects via the UI:^[3]

1. Press `Cmd + Shift + J` to open Cursor Settings
2. Go to **Rules for AI** under the Cursor Settings section
3. Paste the rules content there and save^[4]

The downside: global rules apply to *every* project, not just ReplyGuard. Option 1 is better for project-specific rules.^[5]

Option 3 — Reference Manually in Chat

If you're already mid-session and want to load the rules immediately without restarting:^[1]

Type this at the top of your chat:

```
@replyguard (or @rules if Cursor indexes it)
```

Or just paste: *"For the remainder of this session, remember the following rules: [paste rules]"* — Cursor will confirm it understood.^[1]

Go with Option 1. Create the folder, drop the file, commit it with your `PRD.md` and `BUILD_PLAN.md`, then paste Prompt 0. The agent will have the full constraint set in context from turn one.^[2]



1. <https://github.com/squirrellogic/cursor-rules>
2. <https://blink.new/blog/cursor-rules-guide>
3. <https://kirill-markin.com/articles/cursor-ide-rules-for-ai/>
4. <https://forum.cursor.com/t/can-anyone-help-me-use-this-new-cursor-rules-functionality/45692>
5. <https://mer.vin/2025/12/cursor-ide-rules-deep-dive/>

6. <https://dev.to/yigit-konur/complete-guide-how-to-set-ai-coding-rules-for-cursor-ff2>
7. https://github.com/DVC2/cursor_prompts
8. <https://github.com/paulpham157/paul-s-cursor-rules>
9. https://github.com/Mawla/cursor_rules
10. <https://forum.cursor.com/t/cursor-setup-with-rules-mdc-agents-md-and-hooks/161005>
11. <https://medium.com/@emir.anyig/cursor-project-rules-guide-6c55163421b2>
12. <https://www.youtube.com/watch?v=gw8otRr2zpw>
13. <https://www.youtube.com/watch?v=6vtBq6TBGyo>
14. <https://design.dev/guides/cursor-rules/>
15. https://www.reddit.com/user/Kirmark/comments/1ky3tg/ultimate_cursor_ai_ide_rules_guide_all_5_levels/